Don't update my code or anything, explain what the problem is. I am saving the graph created using NumPY and MatPlotLib, and updating the state class so it shows that on the page in Drafter, however, an image is not showing properly, it shows some broken image.

PLease explain to me what I can do to fix this?

```
from dataclasses import dataclass, field
import time
import drafter
from drafter import *
import matplotlib.pyplot as plt
import numpy as np
import serial

# Stores one sensor reading
@dataclass
class SensorReading:
    timestamp: float
    distance: float

# Stores all the information for the program
@dataclass
class State:
    duration: float = 5.0
    status_message: str = "Ready to collect data."
    readings: list = field(default_factory=list)
    graph_path: str = ""

# Calculates the slope and intercept of the data
def calculate_linear_regression(
    time_arr: list[float], dist_arr: list[float]
) -> tuple[float, float]:

    t = np.array(time_arr)
    d = np.array(dist_arr)

    m, b = np.polyfit(t, d, 1)

    return m, b

# HOME PAGE
@route("/")
def index(state: State) -> Page:
```

```
    return Page(
        state,
        [
            Header("Arduino Distance Logger", level=1),
            Paragraph("Welcome to the sensor telemetry dashboard."),
            Link("Configure Session", "/config"),
        ],
    )

# CONFIGURATION PAGE
@route("/config")
def config_page(state: State) -> Page:

    return Page(
        state,
        [
            Header("Session Setup", level=1),
            Paragraph("Enter the duration in seconds:"),
            TextBox("duration", str(state.duration)),
            Button("Run Logger", "/run"),
            Link("Back To Home", "/"),
        ],
    )

# DATA COLLECTION PAGE
@route("/run")
def run_logger(state: State, duration: str) -> Page:
    state.duration = float(duration)

    # Make sure the duration is positive
    if state.duration <= 0:
        state.duration = 5.0

    start_time = float(time.time())

    # Try connecting to the Arduino
    try:
        arduino = serial.Serial("COM5", 9600)

    except Exception as error:
        arduino = None
        print("Arduino connection failed. Using simulation.")

    # Lists to store the data
    time_array = []
    distances = []
```

```
# Check if the Arduino is connected
if arduino is None or not arduino.is_open:

    # Simulation if Arduino is not connected
    time.sleep(1)

    for i in range(1, int(state.duration) + 1):

        time_array.append(float(i))
        distances.append(12.5 + (i * 1.8))

else:

    # Collect data from the Arduino
    for i in range(1, int(state.duration) + 1):

        target_time = start_time + i

        while time.time() < target_time:
            time.sleep(0.01)

        raw_data = arduino.readline()

        if raw_data:

            try:
                distance_str = raw_data.decode("utf-8").strip()

                if distance_str:
                    distances.append(float(distance_str))
                    time_array.append(float(i))

            except ValueError:
                continue

    arduino.close()

# Make sure there are enough readings for a graph
if len(distances) < 2:

    time_array = [1.0, 2.0, 3.0]
    distances = [10.0, 14.0, 18.0]

# Store the readings
state.readings = []

for t, d in zip(time_array, distances):
```

```
    state.readings.append(
        SensorReading(timestamp=t, distance=d)
    )

# Calculate the linear regression
m, b = calculate_linear_regression(time_array, distances)

predicted_line = []

for t in time_array:
    predicted_line.append(m * t + b)

# Create the graph
plt.figure()

plt.scatter(
    time_array,
    distances,
    color="blue",
    label="Data"
)

plt.plot(
    time_array,
    predicted_line,
    color="red",
    linestyle="--",
    label="Fit"
)

plt.xlabel("Time (s)")
plt.ylabel("Distance (cm)")
plt.legend()

# Save the graph
state.graph_path = "graph.png"

plt.savefig(state.graph_path)
plt.close()
print("Graph saved successfully!")

# Update the status message
state.status_message = (
    f"Collected {len(state.readings)} points successfully!"
)
```

```
    return results_page(state)

# RESULTS PAGE
@route("/results")
def results_page(state: State) -> Page:

    # Create a paragraph for each reading
    reading_paragraphs = []

    for reading in state.readings:

        text = (
            f"Time: {reading.timestamp}s | "
            f"Distance: {reading.distance:.2f} cm"
        )

        reading_paragraphs.append(Paragraph(text))

    return Page(
        state,
        [
            Header("Results & Graph", level=1),
            Paragraph(state.status_message),

            Image(state.graph_path),

            Header("Recorded Data Log", level=2),

            *reading_paragraphs,

            Link("Run Again", "/config"),
            Link("Home", "/"),
        ],
    )

# HISTORY PAGE
@route("/history")
def history_page(state: State) -> Page:

    return Page(
        state,
        [
            Header("History", level=1),
            Paragraph(
                f"Total readings currently in state: {len(state.readings)}"
            ),
```

```
            Link("Home", "/"),
        ],
    )

# ABOUT PAGE
@route("/about")
def about_page(state: State) -> Page:

    return Page(
        state,
        [
            Header("About", level=1),
            Paragraph(
                "Arduino sensor project built with Drafter and Python."
            ),
            Link("Home", "/"),
        ],
    )

# RESET PAGE
@route("/reset")
def reset_state(state: State) -> Page:

    state.duration = 5.0
    state.status_message = "Reset."
    state.readings = []
    state.graph_path = ""

    return index(state)

# START THE PROGRAM
if __name__ == "__main__":
    start_server(State())
```

Your NumPy and Matplotlib code is probably working correctly. I actually think the problem is not the graph generation itself, but how Drafter is trying to display the saved image in the browser.

There are two separate processes happening:

1. Python creates and saves the graph. Matplotlib saves `graph.png` to your computer's filesystem.
2. Drafter displays the graph. The browser needs to be able to access that image through a URL or a file location that Drafter serves.

These are not automatically the same thing.

1. The main problem: `graph.png` is a local file

Look at these two lines:

```
None
state.graph_path = "graph.png"plt.savefig(state.graph_path)
```

The first line stores the filename in your State class.

The second line tells Matplotlib to save the image.

So far, that's fine. In fact, you may even find `graph.png` in your project folder.

However, when you do:

```
None
Image(state.graph_path)
```

Drafter needs to turn that image reference into something the browser can actually retrieve.

The problem is that saving an image on your computer does not automatically make it accessible to the browser.

Think of it this way:

- Python knows where `graph.png` is.
- Your computer knows where `graph.png` is.
- But the browser doesn't necessarily have permission or a route to access that file.

That's why you can get a broken-image symbol even when Matplotlib successfully saves the graph.

2. What Drafter expects

According to the Drafter documentation, `Image()` can accept several types of inputs:

- A URL pointing to an image.
- A local image filename.
- A `Picture` object.
- Raw image bytes.
- A PIL image.

One important distinction is that a local filename and an image's actual contents are different things.

Drafter Documentation+1

For example, imagine you have:

```
None
state.graph_path = "graph.png"
```

This is just a string containing the filename. It isn't the actual image data.

Drafter needs to resolve that filename correctly when it renders the page.

3. Three things you can investigate

A. Where is Matplotlib saving the image?

Your code uses:

```
None
plt.savefig(state.graph_path)
```

Because you're using a relative path, Python saves the file relative to its current working directory.

That might not be the same directory you expect.

Check your project folder and confirm that `graph.png` actually exists after running the logger.

B. Is Drafter able to access the file?

Even if the file exists, the browser might not be able to retrieve it.

One way to investigate is to open your browser's developer tools:

1. Press F12.
2. Open the Network tab.
3. Reload the results page.
4. Look for a request involving `graph.png`.

Pay attention to the status:

- `200`: The browser successfully retrieved the image.
- `404`: The browser couldn't find the image at the requested location.
- `403`: Access to the image was denied.

A 404 would be particularly useful evidence that this is a file-location or serving problem.

C. Is the image itself valid?

Try opening `graph.png` directly from your project folder.

If it opens normally, that tells you Matplotlib successfully generated the image. You can then focus on the connection between the image file and Drafter.

4. Two possible solutions

You have two approaches worth considering.

Option 1: Use Drafter's `Picture` type

Drafter has a dedicated image data type called `Picture`.

Instead of relying on a filename being interpreted correctly by the browser, you can load the saved PNG into a `Picture` object.

Conceptually, the process becomes:

Matplotlib

Generates the graph

PNG file

Saves the graph to your computer

Drafter Picture

Loads the actual image data

Drafter Image

Displays the image on your results page

You can read about this in the Drafter Picture documentation.

Option 2: Use Drafter's `MatPlotLibPlot`

Something particularly interesting: Drafter actually has a component specifically designed for displaying Matplotlib graphs.

It's called:

```
None
MatPlotLibPlot()
```

It captures the current Matplotlib figure and embeds it directly into your webpage.

This means you potentially don't need to save the graph as a separate PNG file at all.

See the MatPlotLibPlot documentation.

5. What I recommend for your project

I'd investigate Option 1 first because you're already successfully generating and saving PNG files.

It also teaches you something useful about web development: the difference between a file path, an image object, and an image displayed by a browser.

One additional thing: your `State` class currently stores:

```
None
graph_path: str = ""
```

That stores a filename, not the image itself. Drafter supports storing `Picture` objects in state, too.

You don't necessarily need to change your entire State class, though.

My question: When you run your program, can you actually open `graph.png` from your project folder and see the graph normally?

That will help us distinguish between a Matplotlib problem and a Drafter image-display problem.

I opened my folder i don't see anything.

Okay, that's an important clue. If you don't see `graph.png` in your project folder, we need to investigate the file-saving process before worrying about Drafter.

Look at this line:

```
None
plt.savefig(state.graph_path)
```

You're saving the graph using a relative path:

```
None
state.graph_path = "graph.png"
```

This means Python saves the file in its current working directory, which isn't necessarily the folder containing your Python script.

Step 1: Find where Python is saving it

Temporarily add these two lines immediately before `plt.savefig()`:

```
None
import osprint(os.getcwd())
```

Don't change anything else.

Step 2: Run your program

1. Start your Drafter server.
2. Run the logger.
3. Look at the VS Code terminal.
4. Find the directory printed by `os.getcwd()`.

That is the directory where Python is attempting to save `graph.png`.

Step 3: Check that directory

Open the printed directory in File Explorer and look for `graph.png`.

What we're testing

- If `graph.png` exists there, Matplotlib is saving successfully, and we can focus on Drafter displaying it.
- If it doesn't exist, we need to investigate why the save operation isn't producing the file.

Don't rewrite your code yet. Let's identify exactly where the image is going first.

drafter console after adding that for debugging purposes:

Pillow already loaded from default channel
Loading Jinja2, MarkupSafe, contourpy, cycler, fonttools, kiwisolver, matplotlib, numpy, packaging, pyparsing, python-dateutil, pytz, six
Loaded Jinja2, MarkupSafe, contourpy, cycler, fonttools, kiwisolver, matplotlib, numpy, packaging, pyparsing, python-dateutil, pytz, six
matplotlib already loaded from default channel
numpy already loaded from default channel
No new packages to load
Loading future, pyyaml
Loaded future, pyyaml
Arduino connection failed. Using simulation.
/home/pyodide
False
main.py:179: MatplotlibDeprecationWarning: The x parameter as float was deprecated in Matplotlib 3.10 and will be removed in 3.12. Use int(x) instead.
plt.savefig(state.graph_path)
main.py:179: MatplotlibDeprecationWarning: The y parameter as float was deprecated in Matplotlib 3.10 and will be removed in 3.12. Use int(y) instead.
plt.savefig(state.graph_path)
Graph saved successfully!
Worked for 7s

Clarifying the file path

OH, this explains a lot. Your console output reveals something important:

```
None
/home/pyodide
False
...
Graph saved successfully!
```

You're running Python inside a Pyodide environment, not directly on your Windows computer.

That's the key discovery.

1. What is `/home/pyodide`?

Pyodide allows Python to run inside a browser-based environment using WebAssembly.

This means:

- Your Python code runs in a virtual filesystem.
- Files saved by Python may exist inside that virtual filesystem.
- They don't necessarily appear in your Windows project folder.
- Windows File Explorer cannot automatically access those virtual files.

So when you save:

```
None
plt.savefig("graph.png")
```

Matplotlib saves the image relative to `/home/pyodide`, not necessarily your Windows project directory.

2. What does `False` mean?

I'm assuming you also added something like:

```
None
print(os.path.exists("graph.png"))
```

If so, `False` means the file didn't exist at the moment that check was performed.

If you placed that check BEFORE `plt.savefig()`, that's completely normal!

You would want to check again AFTER saving.

3. What about the Matplotlib warnings?

These messages:

```
None
MatplotlibDeprecationWarning
```

are warnings about how Matplotlib handles certain arguments internally.

They are not the reason your image isn't displaying.

Your graph generation appears to get past `savefig()` without an exception.

4. What I want you to investigate next

Before changing your code, I want to establish whether the image actually exists inside Pyodide.

Immediately AFTER:

```
None
plt.savefig(state.graph_path)
```

temporarily add:

```
None
import
osprint(os.path.exists(state.graph_path))print(os.path.abspath(stat
e.graph_path))print(os.listdir("."))
```

This will tell us:

1. Whether the image exists after saving.
2. Its absolute path.
3. Whether `graph.png` appears in the current virtual directory.

Here's the bigger picture: I suspect your issue is the interaction between Pyodide's virtual filesystem and Drafter's image-rendering system.

Once we establish that the PNG exists, we can work on getting Drafter to display it without unnecessarily rewriting your project.

I wouldn't abandon your current approach yet. You're actually quite close.

Done.

Pillow already loaded from default channel
Loading Jinja2, MarkupSafe, contourpy, cycler, fonttools, kiwisolver, matplotlib, numpy, packaging, pyparsing, python-dateutil, pytz, six
Loaded Jinja2, MarkupSafe, contourpy, cycler, fonttools, kiwisolver, matplotlib, numpy, packaging, pyparsing, python-dateutil, pytz, six
matplotlib already loaded from default channel
numpy already loaded from default channel
No new packages to load
Loading future, pyyaml

Loaded future, pyyaml
Arduino connection failed. Using simulation.
/home/pyodide
False
main.py:179: MatplotlibDeprecationWarning: The x parameter as float was deprecated in Matplotlib 3.10 and will be removed in 3.12. Use int(x) instead.
plt.savefig(state.graph_path)
main.py:179: MatplotlibDeprecationWarning: The y parameter as float was deprecated in Matplotlib 3.10 and will be removed in 3.12. Use int(y) instead.
plt.savefig(state.graph_path)
True
/home/pyodide/graph.png
['.matplotlib', 'graph.png']
Graph saved successfully!

Perfect. Now we've isolated the problem significantly.

Your latest output confirms:

```
None
/home/pyodide
False
...
True
/home/pyodide/graph.png
['.matplotlib', 'graph.png']
```

Here's what this tells us:

Test	Result	Meaning
Before saving	`False`	Image didn't exist yet. Normal.
After saving	`True`	Matplotlib successfully created the image.

Absolute path	`/home/pyodide/graph.png`	We know exactly where it is.
Directory listing	`graph.png`	The file exists in Pyodide's filesystem.

Confirmed
Your Matplotlib graph is being generated and saved successfully.

So what's actually wrong?

We're now looking at the connection between Pyodide, Drafter, and the browser.

Your code does this:

1. Matplotlib creates `graph.png`.
2. Pyodide stores it in its virtual filesystem.
3. Drafter receives the filename through `Image(state.graph_path)`.
4. The browser attempts to display the image.

The potential failure is step 3 or 4.

The fact that the file exists inside Pyodide doesn't guarantee that Drafter's `Image()` component can serve it to the browser using that filename.

One more test

I want to check whether Drafter can interpret the image independently of your State class.

Temporarily, in your `results_page`, try displaying a known image URL instead of your graph:

```
None
Image("https://placehold.co/600x400/png")
```

Don't change anything else.

What we're testing:

- If the placeholder image displays, Drafter's Image component works, and our problem is specifically how the locally generated PNG is being passed to Drafter.

- If the placeholder also breaks, we need to investigate the Image component or your Drafter environment itself.

This is a useful debugging technique: isolate individual components instead of changing several things simultaneously.

Let me know what happens. We're narrowing this down systematically.

i went into the drafter documentation and found this, Image(Picture.from_file(state.graph_path))

and it worked! Thank you!

LET'S GO!! That's exactly the issue we were trying to isolate.

You found the solution:

```
None
Image(Picture.from_file(state.graph_path))
```

Here's why it works:

- `state.graph_path` is just a string containing `"graph.png"`.
- `Picture.from_file()` reads the actual image from Pyodide's filesystem.
- `Image()` receives a proper Drafter `Picture` object that it can display.

The important distinction:

`Image()` needs an image it can render, not simply a filename that exists in your virtual filesystem.

You also learned something valuable about debugging:

1. Verify that the file exists.
2. Identify the environment where it's saved.
3. Isolate the failing component.
4. Consult the library documentation.
5. Test the appropriate data type.

Honestly, finding that solution yourself in the Drafter documentation is a great step. You're learning how to debug unfamiliar libraries rather than just relying on code examples.

Good work!